

---

## CRYPTOLOGY LAB ASSIGNMENT 02

### Importance of Cryptography: Capturing Dummy Login Credentials over HTTP and HTTPS with Wireshark

|                |                    |
|----------------|--------------------|
| <b>Name</b>    | sivaramakrishnan R |
| <b>Reg. No</b> | 261100690026       |
| <b>Date</b>    | 19-Aug-26          |

#### What I did in this?

Two versions of the same dummy login were tried out on this machine: one that travels as plain text over HTTP, and one that travels wrapped in TLS over HTTPS. The username and password are identical in both cases, only the transport changes, and Wireshark sits on the wire in each run to record exactly what a person watching the network would actually get to see. The point of the exercise is to turn “HTTPS is more secure” from something read in a textbook into something read off a packet capture, and to use that evidence to talk about what TLS is really doing for a login form.

The setup went through two stages. The first stage used a proper two-machine layout with Apache and PHP handling the login, which is closer to how a real site is deployed. Once that confirmed the basic idea, the second stage moved to a single small Python script that serves the same login form over HTTP and HTTPS side by side on one machine, which made it much easier to run the two captures back to back under identical conditions and line them up for comparison.

---

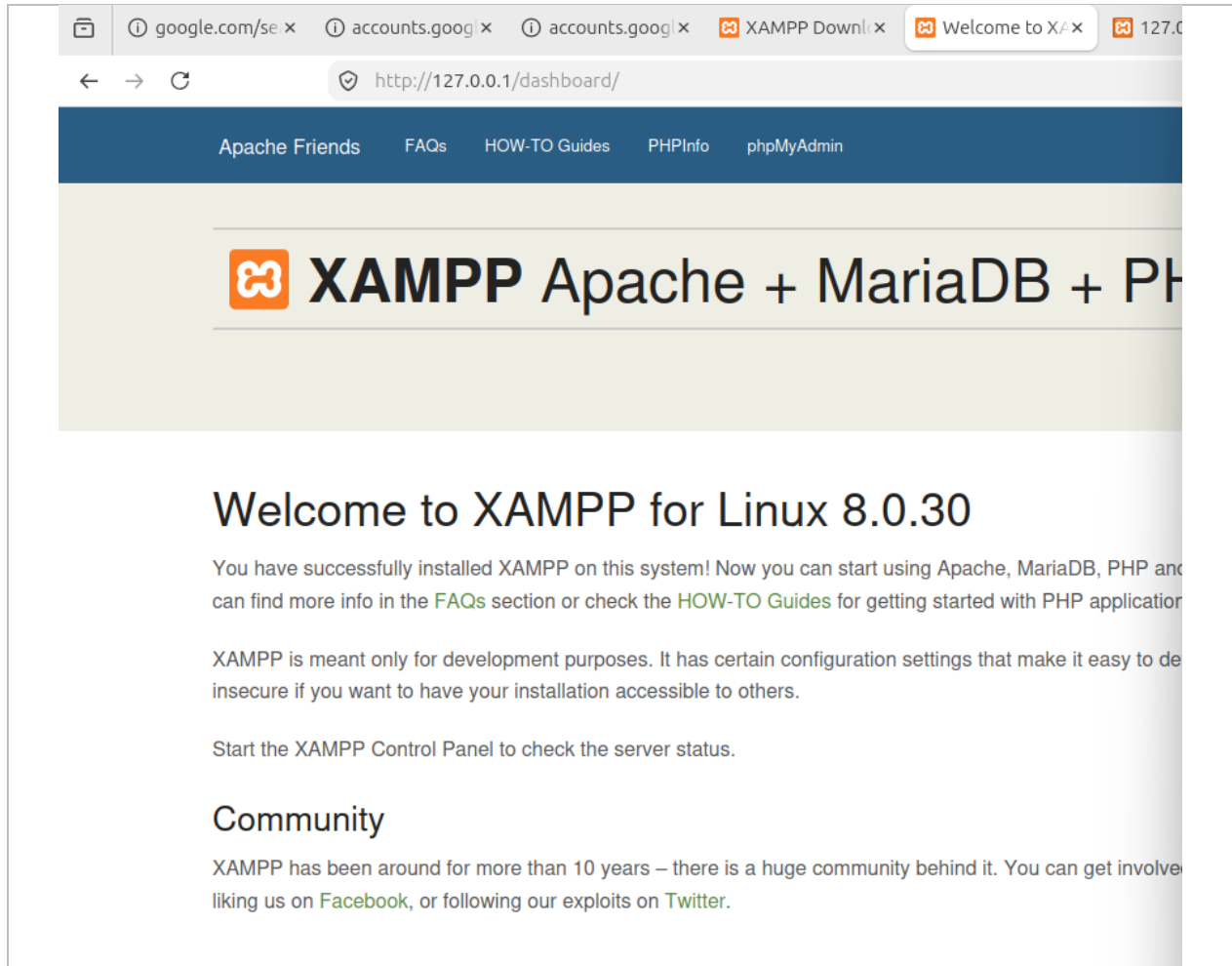
## XAMPP (installation, Server enabling)

XAMPP for Linux was installed on the target machine to get Apache, MariaDB and PHP running quickly. After a couple of false starts — the installer needed execute permission before it would run, and the XAMPP control script had to be launched with sudo — the stack came up cleanly. A cryptolab folder was created under /opt/lampp/htdocs, and index.html and login.php were written into it with nano: a plain HTML form posting to a PHP page that simply echoes back whatever username and password it receives, enough to prove that credentials travelling in the request can be read on the other end.

```
siva@linux:~/crypto$ cd ..
siva@linux:~$ cd Downloads
siva@linux:~/Downloads$ sudo ./xampp-linux-*.run
[sudo: authenticate] Password:
sudo: './xampp-linux-*.run': command not found
siva@linux:~/Downloads$ nano login.html
siva@linux:~/Downloads$ sudo ./xampp-linux-*.run
[sudo: authenticate] Password:
sudo: cannot execute '/home/siva/Downloads/xampp-linux-x64-8.0.30-0-installer.run': Permission denied (os
siva@linux:~/Downloads$ sudo ./xampp-linux-*.run
Ssudo: cannot execute '/home/siva/Downloads/xampp-linux-x64-8.0.30-0-installer.run': Permission denied (os
siva@linux:~/Downloads$ chmod +x xampp-linux-x64-*-installer.run
siva@linux:~/Downloads$ sudo ./xampp-linux-*.run
siva@linux:~/Downloads$ sudo /opt/lampp/lampp start
[sudo: authenticate] Password:
Starting XAMPP for Linux 8.0.30-0...
XAMPP: Starting Apache...already running.
XAMPP: Starting MySQL.../opt/lampp/share/xampp/xamplib: line 22: netstat: command not found
ok.
XAMPP: Starting ProFTPD.../opt/lampp/share/xampp/xamplib: line 22: netstat: command not found
ok.
siva@linux:~/Downloads$ sudo /opt/lampp/lampp start
Starting XAMPP for Linux 8.0.30-0...
XAMPP: Starting Apache...already running.
XAMPP: Starting MySQL...already running.
XAMPP: Starting ProFTPD...already running.
siva@linux:~/Downloads$ /opt/lampp/htdocs/
bash: /opt/lampp/htdocs/: Is a directory
siva@linux:~/Downloads$ cd /opt/lampp/htdocs
siva@linux:/opt/lampp/htdocs$ sudo mkdir cryptolab
siva@linux:/opt/lampp/htdocs$ cd /opt/lampp/htdocs/cryptolab
siva@linux:/opt/lampp/htdocs/cryptolab$ sudo nano index.html
siva@linux:/opt/lampp/htdocs/cryptolab$ sudo nano login.php
```

**Figure 1:** Setting up XAMPP on the target machine and creating the cryptolab folder with index.html and login.php.

Before wiring up the login page, the XAMPP welcome page was loaded in the browser to confirm Apache, MariaDB and PHP were all actually running rather than just reporting success in the terminal.

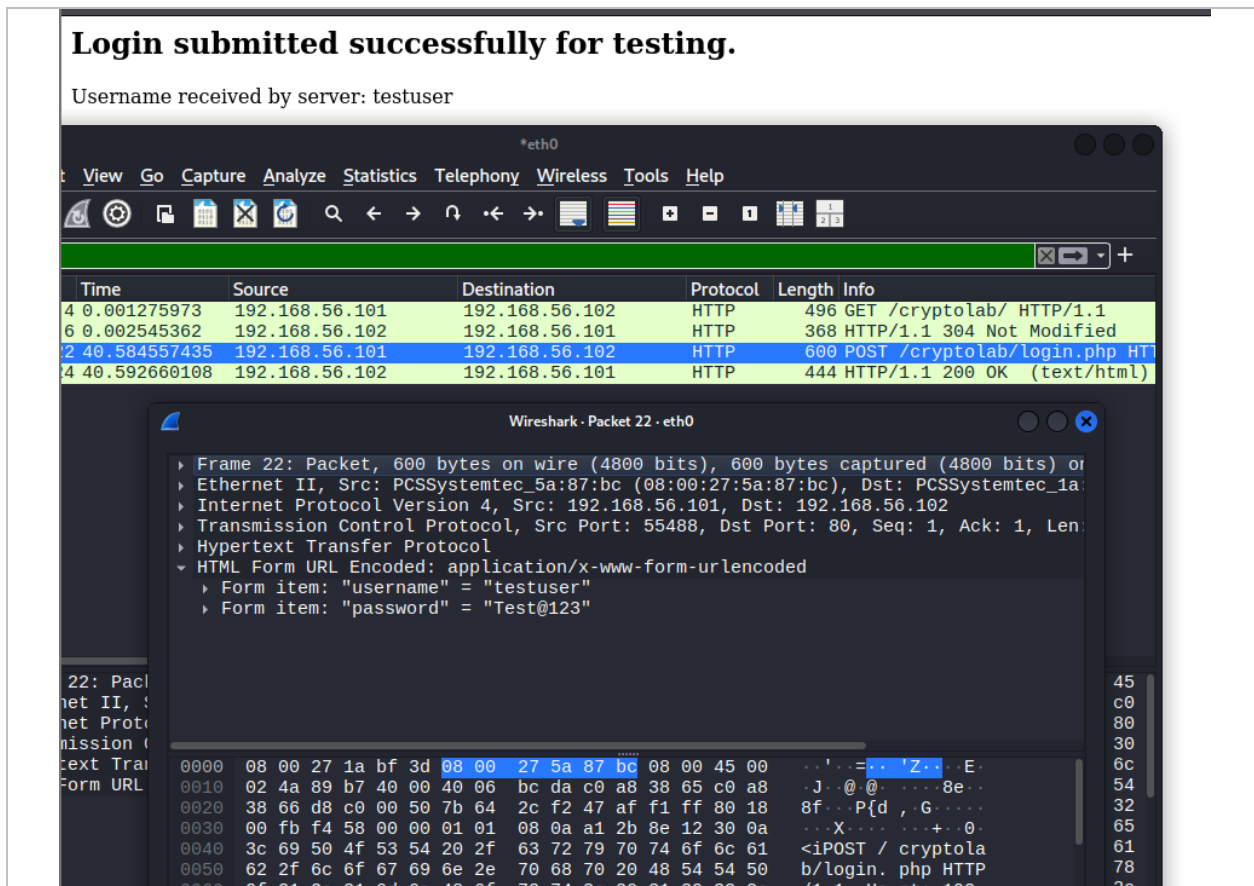


**Figure 2:** The XAMPP welcome page confirms Apache, MariaDB and PHP are live before the login page is tested.

For the actual HTTP-versus-HTTPS comparison, a small Python server (server.py) was used instead. It serves the same login form on two ports of the loopback address at once: plain HTTP on 127.0.0.1:8000, and TLS on 127.0.0.1:8443 using a self-signed certificate and key generated for the lab (cert.pem and key.pem, sitting alongside login.html and server.py). Keeping both protocols on the same script and the same machine means the only variable between the two captures is the transport itself, nothing about the server, the form, or the credentials changes.

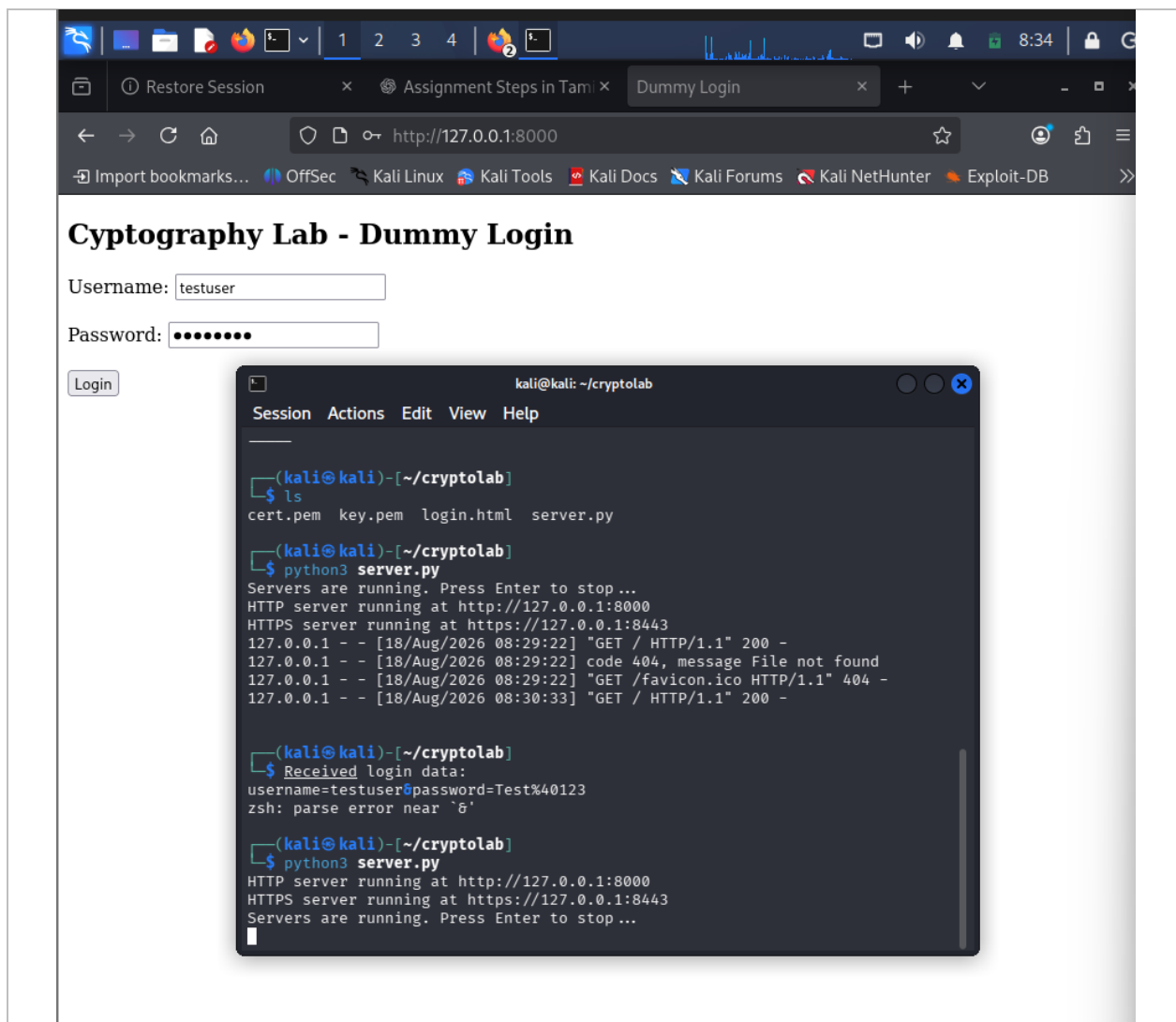
Credentials used throughout: username testuser, password Test@123.

For the two-machine run, Wireshark was left capturing on eth0 while the login page was loaded from the second machine and the form submitted. The capture list shows the GET for /cryptolab/, an HTTP 304 for a cached follow-up request, and then the POST to /cryptolab/login.php that carries the credentials, followed by the page confirming the username the server received.

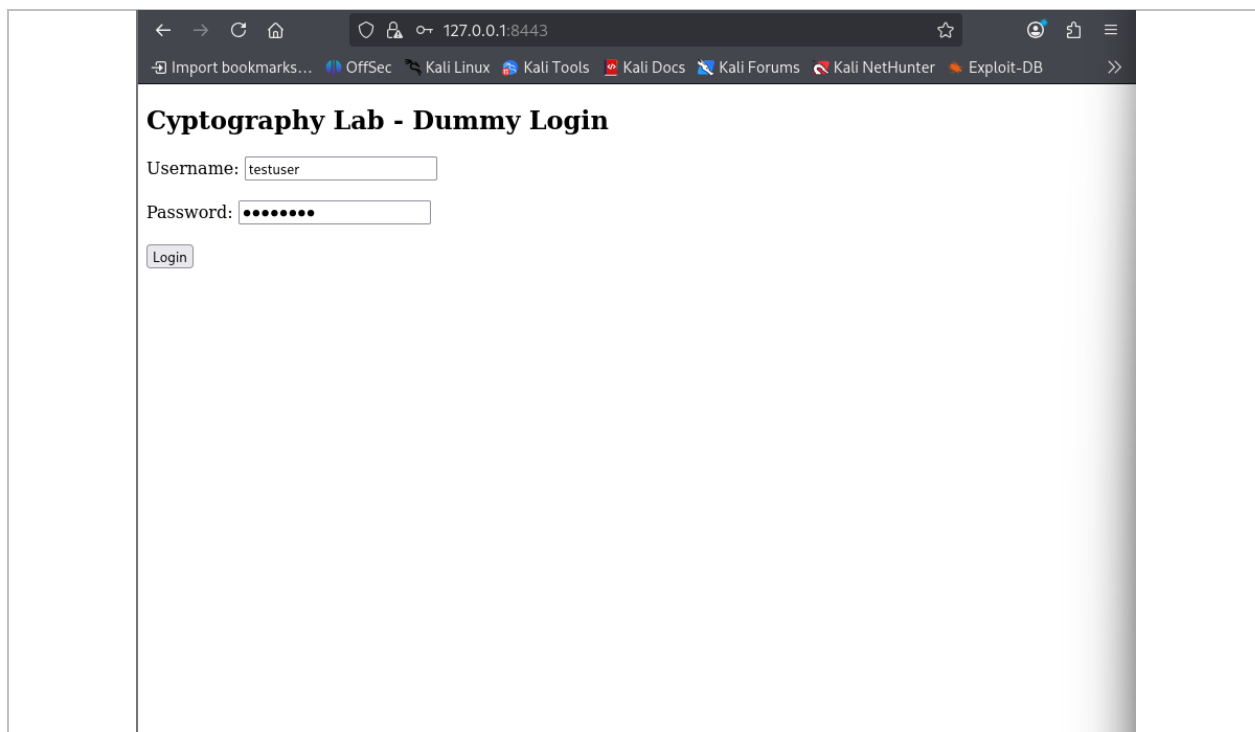


**Figure 3:** Wireshark on eth0 capturing the GET and POST to /cryptolab/, alongside the PHP page confirming the username testuser was received by the server.

For the local run, Wireshark was pointed at the loopback interface (lo) instead, since both client and server sit on the same machine. server.py was started from the cryptolab folder, and the ls output confirms cert.pem, key.pem, login.html and server.py are all present before the servers come up on ports 8000 and 8443. Submitting the form from Firefox over plain HTTP is what shows up in the terminal as a line of received login data. One submission attempt is also visible failing: typing the curl command with the query string unescaped on the shell (username=testuser&password=Test@123) let zsh treat the & as a background-job operator and throw a parse error, a reminder that the ampersand needs quoting on the command line even though it is a perfectly normal form-encoding character over the wire.



**Figure 4:** Firefox loading the login page over `http://127.0.0.1:8000` while the terminal shows `server.py` running both servers, the directory listing, and a submission attempt that hit a shell quoting error.



**Figure 5:** The same dummy login page, this time loaded over the HTTPS port, 127.0.0.1:8443.

Because the form was resubmitted a few times while testing, the server log shows the same received-data line more than once, each one logging username=testuser&password=Test@123 as it arrived along with the timestamped POST /login entry.

```
(kali@kali)-[~/cryptolab]
$ python3 server.py
Servers are running. Press Enter to stop...
HTTP server running at http://127.0.0.1:8000
HTTPS server running at https://127.0.0.1:8443
127.0.0.1 - - [18/Aug/2026 08:50:39] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [18/Aug/2026 08:50:39] code 404, message File not found
127.0.0.1 - - [18/Aug/2026 08:50:39] "GET /favicon.ico HTTP/1.1" 404 -

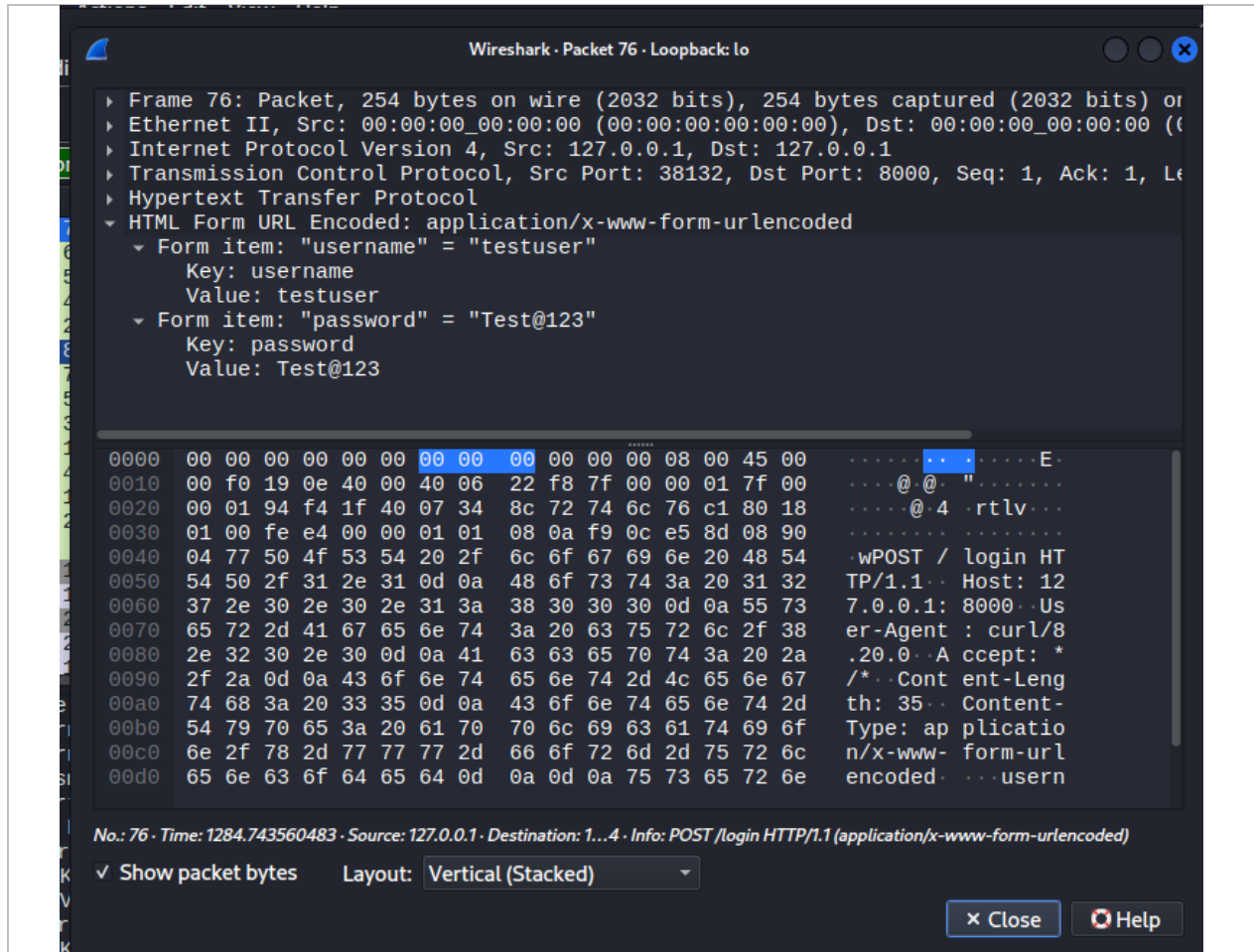
Received login data:
username=testuser&password=Test@123
127.0.0.1 - - [18/Aug/2026 08:54:18] "POST /login HTTP/1.1" 200 -

Received login data:
username=testuser&password=Test@123
127.0.0.1 - - [18/Aug/2026 08:55:52] "POST /login HTTP/1.1" 200 -

Received login data:
username=testuser&password=Test@123
127.0.0.1 - - [18/Aug/2026 09:01:29] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [18/Aug/2026 09:20:55] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [18/Aug/2026 09:20:55] code 404, message File not found
127.0.0.1 - - [18/Aug/2026 09:20:55] "GET /favicon.ico HTTP/1.1" 404 -
```

**Figure 6:** server.py logging repeated form submissions, each one printing the received username and password.

With the capture filtered down to the POST request on port 8000, Wireshark's HTTP dissector parses the request body on its own and lists the two form fields directly: username = testuser and password = Test@123. The hex pane on the right shows the identical characters sitting in the raw bytes of the packet, right after the Content-Type and Content-Length headers. Nothing here needed to be decrypted or reconstructed — the credentials are simply present in the packet exactly as they were typed into the form, along with the curl User-Agent string that sent them, which is itself a small extra detail an eavesdropper gets for free.

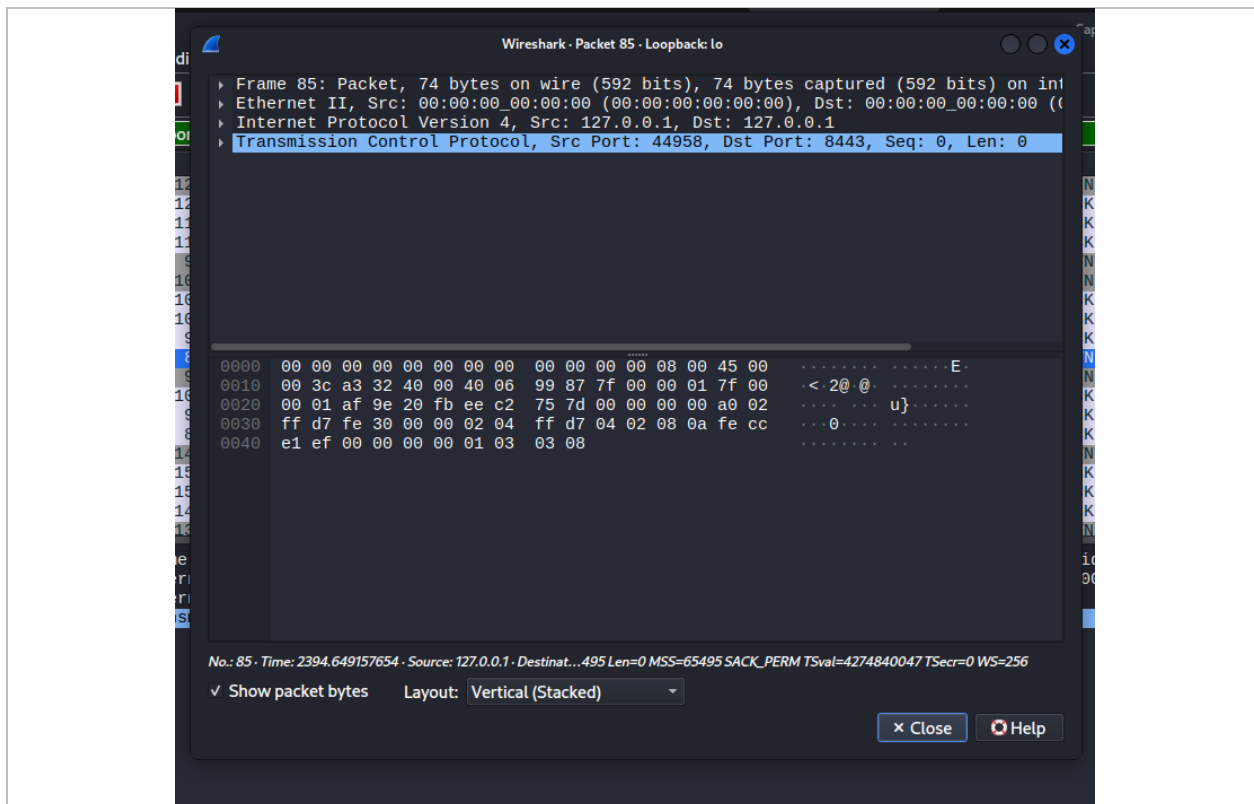


**Figure 7:** The POST /login capture on port 8000. Wireshark decodes the form fields username = testuser and password = Test@123, and the same text is visible in the packet bytes.

## HTTPS Implementing Method

Repeating the exact same submission against port 8443 changes what ends up in the capture completely. Wireshark no longer has an HTTP layer to decode at all — the packet below is simply a TCP segment on its way to port 8443, with no form fields, no username, no password anywhere in it. That absence is the point: the TLS handshake and the encrypted application data that follow give a sniffer nothing to parse. The browser padlock next to 127.0.0.1:8443 on the login page is the visible half of the same story; the

certificate is self-signed for this lab, so the browser will not vouch for the server's identity the way it would for a certificate from a trusted authority, but the channel is encrypted regardless.



**Figure 8:** A packet destined for port 8443 on the loopback interface. Only TCP fields are visible — there is no HTTP layer for Wireshark to decode, since the payload travels inside TLS.

The only thing that changed between the two captures is the scheme in the URL, yet one leaves a password sitting in plain sight and the other leaves nothing usable at all. TLS is doing three separate jobs here, even though the lab only really exercises the first one directly. It keeps the payload confidential, which is exactly what hides the password in the port 8443 capture. It also protects integrity, so any tampering with the ciphertext in transit would be caught rather than silently accepted. And in a normal deployment it authenticates the server through a certificate signed by a trusted authority, which is what stops an attacker from standing in as the site. This lab's certificate is self-signed, so the browser flags the identity as unverified, but that warning is about trust in who is on the other end, not about the strength of the encryption itself — the channel is still opaque to a sniffer either way.

Plain HTTP, by contrast, trusts every device the traffic happens to pass through: the local switch, a shared Wi-Fi access point, an upstream router, an ISP. Any one of them can read a password sent that way, and the port 8000 capture in this lab is a small-scale demonstration of exactly that. Moving the same form to HTTPS takes that trust away from the network path and puts it in the session keys instead, which is the entire reason a login page has no business running over plain HTTP.



---

The two captures make the difference concrete rather than theoretical. Over HTTP, testuser and Test@123 come straight out of the captured POST packet, decoded by Wireshark's own HTTP dissector with no extra effort. Over HTTPS, the identical login produces a packet with nothing but TCP and TLS to look at, and the credentials never appear anywhere in the capture. Encryption is what closes that gap, and having actually watched a password sit in the clear on one port and disappear on the other is a far more convincing reason to insist on HTTPS than any line in a textbook.